EXP 25

```
lc;
clear;
close all;

% Simulation of Convolutional Coding
% and Analysis of Encoded Output Bit Sequences

% Step 1: Parameters
N = 20;                 % Number of input bits
constraint_length = 3;
memory = constraint_length - 1;

% Generator polynomials: 111 and 101
g1 = [1 1 1];
g2 = [1 0 1];

% Step 2: Generate Random Input Bits
data = randi([0 1], 1, N);

% Add tail bits to return encoder to zero state
input_bits = [data zeros(1, memory)];

% Step 3: Convolutional Encoding
state = zeros(1, memory);
encoded = [];
```

```
for i = 1:length(input_bits)

    reg = [input_bits(i) state];

    out1 = mod(sum(reg .* g1), 2);
    out2 = mod(sum(reg .* g2), 2);

    encoded = [encoded out1 out2];

    state = reg(1:memory);
end

% Step 4: Display Input and Encoded Bits
disp('Original Input Bits:');
disp(data);

disp('Input Bits with Tail Bits:');
disp(input_bits);

disp('Convolutionally Encoded Output Bits:');
disp(encoded);

% Step 5: BPSK Modulation
tx = 1 - 2*encoded;
```

```
% Step 6: Add AWGN
SNR_dB = 4;
signal_power = mean(tx.^2);
noise_power = signal_power / (10^(SNR_dB/10));

rx = tx + sqrt(noise_power)*randn(size(tx));

% Step 7: Hard Decision Detection
received_bits = rx < 0;

disp('Received Bits after Hard Decision:');
disp(received_bits);

% Step 8: Viterbi Decoding
num_states = 2^memory;
num_steps = length(received_bits)/2;

path_metric = inf(num_states, num_steps+1);
path_metric(1,1) = 0;

previous_state = zeros(num_states, num_steps);
previous_input = zeros(num_states, num_steps);

for t = 1:num_steps

    r = received_bits(2*t-1:2*t);
```

```
for s = 0:num_states-1

    if isinf(path_metric(s+1,t))
        continue;
    end

    % Convert state to two memory bits
    state_bits = [bitget(s,2), bitget(s,1)];

    for u = 0:1

        reg = [u state_bits];

        o1 = mod(sum(reg .* g1), 2);
        o2 = mod(sum(reg .* g2), 2);
        out = [o1 o2];

        % Next state
        next_bits = reg(1:memory);
        ns = next_bits(1)*2 + next_bits(2);

        % Hamming distance
        distance = sum(out ~= r);

        new_metric = path_metric(s+1,t) + distance;
```

```
            if new_metric < path_metric(ns+1,t+1)
                path_metric(ns+1,t+1) = new_metric;
                previous_state(ns+1,t) = s;
                previous_input(ns+1,t) = u;
            end
        end
    end
end

% Traceback from zero state because tail bits were added
decoded = zeros(1, num_steps);
current_state = 0;

for t = num_steps:-1:1
    decoded(t) = previous_input(current_state+1,t);
    current_state = previous_state(current_state+1,t);
end

decoded_data = decoded(1:N);

disp('Decoded Output Bits:');
disp(decoded_data);

% Step 9: Calculate Bit Error Rate
errors = sum(data ~= decoded_data);
```

```
BER = errors/N;

fprintf('\nNumber of Bit Errors = %d\n', errors);
fprintf('Bit Error Rate (BER) = %.4f\n', BER);

% Step 10: Plot Input and Encoded Sequences
figure;

subplot(3,1,1);
stairs(data, 'LineWidth', 1.5);
grid on;
ylim([-0.2 1.2]);
title('Original Input Bit Sequence');
xlabel('Bit Index');
ylabel('Bit');

subplot(3,1,2);
stairs(encoded, 'LineWidth', 1.5);
grid on;
ylim([-0.2 1.2]);
title('Convolutionally Encoded Output Sequence');
xlabel('Encoded Bit Index');
ylabel('Bit');

subplot(3,1,3);
stairs(decoded_data, 'LineWidth', 1.5);
```

```
grid on;
ylim([-0.2 1.2]);
title('Decoded Output Bit Sequence');
xlabel('Bit Index');
ylabel('Bit');
```

